



Computational Logic in the Era of AI

The λ -Cube and Propositions-as-Types

Sergey Mechtaev

mechtaev@pku.edu.cn

Term 1 • 26–27

This lecture

↓ you are here

■ theory ■ in Lean



1 Polymorphic

$\lambda 2$: a term takes a type; the binder Π

$\{\alpha : \text{Type}\}$

- One definition works at every type, and its type says so.

2 Constructors

$\lambda \omega$: a type takes a type; conversion

`List, rfl`

- Types can be computed, and types that compute to the same result are interchangeable.

3 Dependency

λP : a type takes a term; $\Pi x:A. B$

`Vec  $\alpha$  n`

- A type can mention a value, so a type can state a fact about that value.

4 Proofs

$\Rightarrow$ is $\rightarrow$, and $\forall$ is Π

`theorem, fun`

- A proof is a term, and checking a proof is checking a type.

5 The cube

λC : one rule set, eight systems

`Prop, Type`

- Checking a proof is decidable. Finding one is not.

History: three extensions, one calculus



N. G. de Bruijn

1918–2012

A proof is a term,
and a small
program checks it.

1968 De Bruijn's Automath: the first system in which mathematics is written down as typed terms and checked by machine.

1968 Howard writes out what Curry had noticed: propositions correspond to types, proofs to terms. Printed in 1980.

1972 Girard's system F, and Reynolds independently in 1974: terms that take types as arguments. Today's $\lambda 2$.

1970s Martin-Löf's type theory: types that depend on terms, as a foundation for constructive mathematics.

1985 Coquand and Huet: the Calculus of Constructions, λC , combines all of these.

1991 Barendregt arranges eight such systems in a cube, under one set of rules.



Thierry Coquand

born 1961

One calculus for
programs, types and
proofs.

Coq and Lean both check proofs in λC extended with inductive types. Photos: K. Jacobs/MFO (CC BY-SA 2.0 de) and A. Bauer (CC BY-SA 2.5 si), via Wikimedia Commons. N&G §3.8, §5.7, §6.2, §6.5

$\lambda \rightarrow$ has a single dependency

In $\lambda x:\sigma. M$ the body M may mention x : the term $\lambda x:\sigma. M$ depends on the term x . This is the only dependency in $\lambda \rightarrow$. With terms and types there are four, and three are missing.

what is built

depending on a **term**

depending on a **type**

a **term**

$\lambda \rightarrow$

last lecture

$\lambda x:\sigma. M$

`fun (n : Nat) => n + 1`

$\lambda 2$

$\lambda \alpha:*. \lambda x:\alpha. x$

`fun (α : Type) (x : α) => x`

a **type**

λP

$\lambda n:\text{nat}. V_n$

`fun (n : Nat) => Vec Nat n`

$\lambda \omega$

$\lambda \alpha:*. \alpha \rightarrow \alpha$

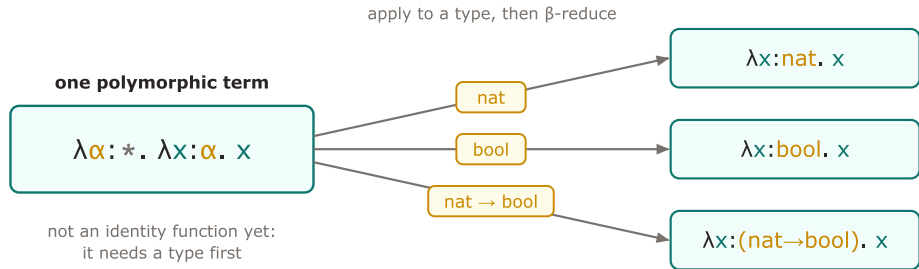
`fun (α : Type) => α → α`

N&G §3.1, §5.1

$\lambda 2$: terms depending on types

In $\lambda \rightarrow$ every type has its own identity function, and no single term is the identity on all types.

$\lambda 2$ adds abstraction over a type variable: a term may take a type $\alpha : *$ as an argument. Here $*$ is the type of all types.



Type abstraction and type application

Definition (second order abstraction and application)

- If M is a term and α a type variable, then $\lambda\alpha:*.M$ is a term: a **term depending on a type**.
- If M is a term and T a type, then MT is a term.
- β -reduction gains a clause: $(\lambda\alpha:*.M)T \rightarrow_{\beta} M[\alpha := T]$.

Example. Applying a function twice, for every type α and every $f : \alpha \rightarrow \alpha$:

$$D \equiv \lambda\alpha:*. \lambda f:\alpha \rightarrow \alpha. \lambda x:\alpha. f(fx)$$

$D \text{ nat} \rightarrow_{\beta} \lambda f:\text{nat} \rightarrow \text{nat}. \lambda x:\text{nat}. f(fx)$ a type goes in, a term of $\lambda \rightarrow$ comes out
 $D \text{ nat } s \rightarrow_{\beta} \lambda x:\text{nat}. s(sx)$ with s the successor: the function $n \mapsto n + 2$

N&G Ex. 3.1.1 (2), Def. 3.4.1, 3.6.2

The type of a polymorphic term needs a new binder

Since $\lambda x:\alpha. x : \alpha \rightarrow \alpha$, a first guess for the polymorphic identity is $* \rightarrow (\alpha \rightarrow \alpha)$.

the guess $\lambda \alpha : *. \lambda x : \alpha. x \quad : \quad * \rightarrow (\alpha \rightarrow \alpha)$ ✗

free: no binder reaches them

the repair $\lambda \alpha : *. \lambda x : \alpha. x \quad : \quad \Pi \alpha : *. \alpha \rightarrow \alpha$ ✓

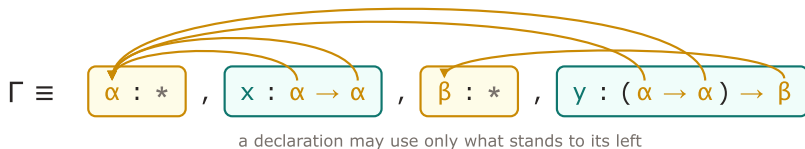
Π binds α inside the type

Renaming the bound α to β gives the same term, but the guessed type changes: one term, two types. With Π , renaming changes both sides alike.

$\Pi \alpha : *. A$ is the type of functions sending a type α to a term of type A . N&G §3.2

Declarations now come in order

In $\lambda 2$ a type variable must be declared before it is used.



Definition ($\lambda 2$ -context)

$\emptyset$ is a $\lambda 2$ -context. If Γ is one, then so are $\Gamma, \alpha : *$ (for α not in Γ) and $\Gamma, x : \rho$ (for x not in Γ , and every free type variable of ρ declared in Γ).

N&G Def. 3.4.4, Ex. 3.4.5

The derivation rules of $\lambda 2$

The types now include Π -types:

$$\mathbb{T}2 = \mathbb{V} \mid (\mathbb{T}2 \rightarrow \mathbb{T}2) \mid (\Pi \mathbb{V} : * . \mathbb{T}2)$$

The three rules of $\lambda \rightarrow$ stay, now over $\lambda 2$ -contexts: (*var*), (*appl*), (*abst*). $\lambda 2$ adds three.

$$\frac{\Gamma, \alpha : * \vdash M : A}{\Gamma \vdash \lambda \alpha : * . M : \Pi \alpha : * . A} \text{ (abst}_2\text{)}$$

$$\frac{\Gamma \vdash M : \Pi \alpha : * . A \quad \Gamma \vdash B : *}{\Gamma \vdash MB : A[\alpha := B]} \text{ (appl}_2\text{)}$$

(*form*) $\Gamma \vdash B : *$ if Γ is a $\lambda 2$ -context, $B \in \mathbb{T}2$, and all free type variables of B are declared in Γ

N&G §3.4, Fig. 3.1

The rule ($abst_2$)

$$\frac{\Gamma, \alpha : * \vdash M : A}{\Gamma \vdash \lambda \alpha : *. M : \Pi \alpha : *. A} \text{ (abst}_2\text{)}$$

The premiss types the body M with $\alpha : *$ in the context. The conclusion moves that declaration out of the context and onto a binder: λ in the term, Π in the type.

Instance

$$\frac{\alpha : * \vdash \lambda x : \alpha. x : \alpha \rightarrow \alpha}{\emptyset \vdash \lambda \alpha : *. \lambda x : \alpha. x : \Pi \alpha : *. \alpha \rightarrow \alpha}$$

The rule (*appl*₂)

$$\frac{\Gamma \vdash M : \Pi\alpha : * . A \quad \Gamma \vdash B : *}{\Gamma \vdash MB : A[\alpha := B]} \text{ (appl}_2\text{)}$$

M expects a type, and B is a type. The type of MB is A with B substituted for α : the type of the result depends on the argument.

■ Instances, with $\Gamma \equiv \text{nat} : *, \text{bool} : *$ and $\text{id} \equiv \lambda\alpha : * . \lambda x : \alpha . x$

$$\frac{\Gamma \vdash \text{id} : \Pi\alpha : * . \alpha \rightarrow \alpha \quad \Gamma \vdash \text{nat} : *}{\Gamma \vdash \text{id nat} : \text{nat} \rightarrow \text{nat}}$$

$$\frac{\Gamma \vdash \text{id} : \Pi\alpha : * . \alpha \rightarrow \alpha \quad \Gamma \vdash \text{nat} \rightarrow \text{bool} : *}{\Gamma \vdash \text{id} (\text{nat} \rightarrow \text{bool}) : (\text{nat} \rightarrow \text{bool}) \rightarrow (\text{nat} \rightarrow \text{bool})}$$

The rule (*form*) of $\lambda 2$

(*form*) $\Gamma \vdash B : *$ if Γ is a $\lambda 2$ -context, $B \in \mathbb{T}2$, and all free type variables of B are declared in Γ

It supplies the second premiss of (*appl*₂): no other rule concludes that something has type $*$.

Instances

$\text{nat} : *, \text{bool} : * \vdash \text{nat} \rightarrow \text{bool} : *$ and $\alpha : * \vdash \Pi \beta : *. \alpha \rightarrow \beta : *$.

Not an instance

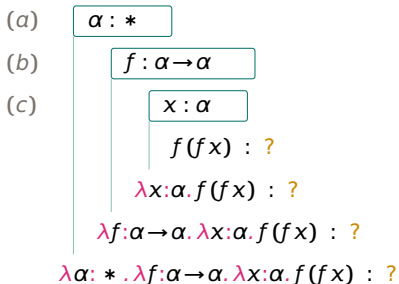
$\alpha : * \vdash \alpha \rightarrow \beta : *$ β is not declared

A typing problem in $\lambda 2$

Problem

Find a type ρ with $\emptyset \vdash D : \rho$, where $D \equiv \lambda\alpha : *. \lambda f : \alpha \rightarrow \alpha. \lambda x : \alpha. f(fx)$.

Bottom to top, each leading λ opens a flag:



a problem of $\lambda \rightarrow$

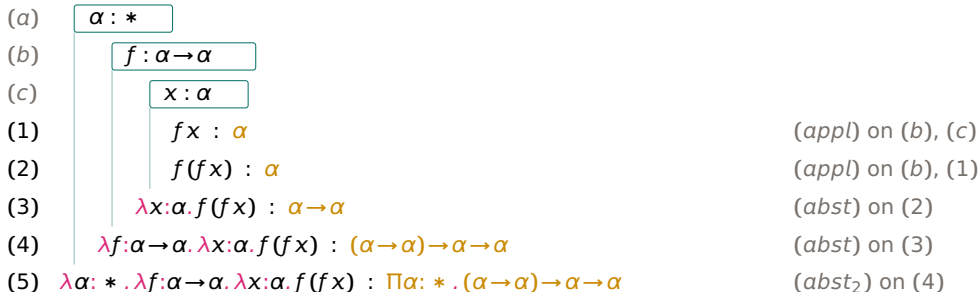
(*abst*)

(*abst*)

(*abst*₂)

N&G §3.5

The derivation, from the inside out



- In (1) and (2), under the three flags, the problem is one of $\lambda \rightarrow$, solved by (appl) twice.
- In (3) to (5), closing a flag puts its declaration in front of the type, as an arrow for x and f , by (abst), and as a Π for α , by (abst₂).

Type abstraction, in Lean

Lean writes `*` as `Type` for data types and as `Prop` for propositions, and $\Pi \alpha : * . A$ as $(\alpha : \text{Type}) \rightarrow A$.

```
--  $\lambda \alpha : *. \lambda x : \alpha. x : \Pi \alpha : *. \alpha \rightarrow \alpha$ 
def idPoly : ( $\alpha : \text{Type}$ )  $\rightarrow \alpha \rightarrow \alpha :=$ 
  fun  $\alpha$  x => x
```

```
#check idPoly Nat           -- idPoly Nat : Nat  $\rightarrow$  Nat
#eval idPoly Nat 3          -- 3
#eval idPoly String "three" -- "three"
```

-- braces: the type argument is still there, but Lean finds it

```
def idImp { $\alpha : \text{Type}$ } (x :  $\alpha$ ) :  $\alpha := x$ 
```

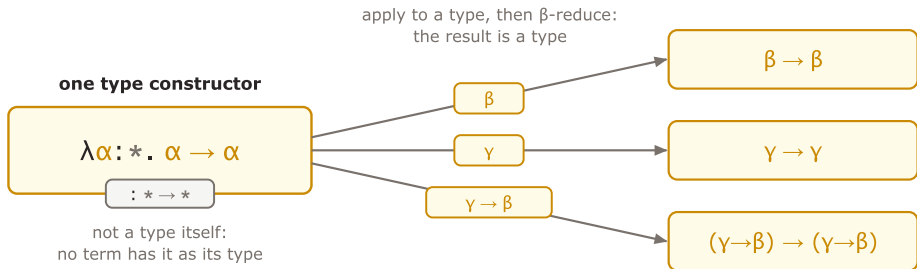
-- '@' switches the braces off, so that the whole type is shown

```
#check @idImp           -- @idImp : { $\alpha : \text{Type}$ }  $\rightarrow \alpha \rightarrow \alpha$ 
#eval idImp 3           -- 3                                 $\alpha := \text{Nat}$ 
```

$\lambda\omega$: types depending on types

The types $\beta \rightarrow \beta$, $\gamma \rightarrow \gamma$, $(\gamma \rightarrow \beta) \rightarrow (\gamma \rightarrow \beta)$ share a pattern.

$\lambda\omega$ adds abstraction over a type variable inside a type. The result is a **type constructor**: a function from types to types.



Kinds

A type has type $*$. A type constructor is not a type, so its type is of another sort: a **kind**.

Definition (kinds)

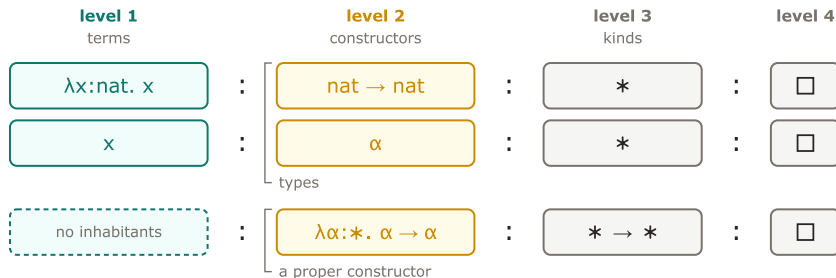
$\mathbb{K} = * \mid (\mathbb{K} \rightarrow \mathbb{K})$, with $\rightarrow$ right-associative, as for types.

constructor	kind	
$\beta \rightarrow \beta$	$*$	a type
$\lambda \alpha: * . \alpha \rightarrow \alpha$	$* \rightarrow *$	takes a type
$\lambda \alpha: * . \lambda \beta: * . \alpha \rightarrow \beta$	$* \rightarrow * \rightarrow *$	takes two types
$\lambda \alpha: * \rightarrow * . \alpha \gamma$	$(* \rightarrow *) \rightarrow *$	takes a constructor, and applies it to $\gamma : *$

All kinds have one type, $\Box$: $* : \Box$, $* \rightarrow * : \Box$, and so on. The symbols $*$ and $\Box$ are the **sorts**; the letter s stands for either.

N&G §4.1

Four levels



Definition (constructor)

If $\kappa : \square$ and $M : \kappa$, then M is a **constructor**; if moreover $\kappa \neq *$, a **proper constructor**. So a constructor is either a type or a proper constructor.

In a statement $A : B$, the level of B is one above the level of A .

N&G Def. 4.1.4, 4.1.6, Rem. 4.1.7

The derivation rules of λ_{ω}

In λ_2 , $\Gamma \vdash B : *$ held by a side condition on the shape of B . Types now compute, so it needs a derivation. The seven rules:

$$\begin{array}{c} \frac{}{\emptyset \vdash * : \square} \text{ (sort)} \quad \frac{\Gamma \vdash A : s}{\Gamma, x:A \vdash x : A} \text{ (var)} \quad \frac{\Gamma \vdash A : B \quad \Gamma \vdash C : s}{\Gamma, x:C \vdash A : B} \text{ (weak)} \\[10pt] \frac{\Gamma \vdash A : s \quad \Gamma \vdash B : s}{\Gamma \vdash A \rightarrow B : s} \text{ (form)} \quad \frac{\Gamma \vdash A : B \quad \Gamma \vdash B' : s}{\Gamma \vdash A : B'} \text{ (conv) if } B =_{\beta} B' \\[10pt] \frac{\Gamma \vdash M : A \rightarrow B \quad \Gamma \vdash N : A}{\Gamma \vdash MN : B} \text{ (appl)} \quad \frac{\Gamma, x:A \vdash M : B \quad \Gamma \vdash A \rightarrow B : s}{\Gamma \vdash \lambda x:A. M : A \rightarrow B} \text{ (abst)} \end{array}$$

s is $*$ or $\square$. In (var) and $(weak)$, x does not occur in Γ .

N&G Fig. 4.1

The rules (*sort*), (*var*) and (*weak*) of $\lambda\omega$

$$\frac{}{\emptyset \vdash * : \square} \text{ (sort)} \quad \frac{\Gamma \vdash A : s}{\Gamma, x:A \vdash x : A} \text{ (var)} \quad \frac{\Gamma \vdash A : B \quad \Gamma \vdash C : s}{\Gamma, x:C \vdash A : B} \text{ (weak)}$$

(*sort*) starts every derivation. (*var*) adds a declaration $x : A$ and derives it, provided A is a type or a kind. (*weak*) keeps what is derivable when the context grows; it is needed because (*var*) derives only the last declaration.

Instances

$$\frac{\frac{\frac{}{\emptyset \vdash * : \square} \text{ (sort)}}{\alpha : * \vdash \alpha : *} \text{ (var), } s \equiv \square}{\frac{\alpha : *, x : \alpha \vdash x : \alpha}{\alpha : * \vdash \alpha : * \quad \alpha : * \vdash \alpha : *} \text{ (var), } s \equiv *}{\alpha : *, x : \alpha \vdash \alpha : *} \text{ (weak), } s \equiv *$$

The rule (form) of $\lambda\omega$

$$\frac{\Gamma \vdash A : s \quad \Gamma \vdash B : s}{\Gamma \vdash A \rightarrow B : s} \text{ (form)}$$

An arrow between two types is a type, and an arrow between two kinds is a kind.

■ Instances, with $\Gamma \equiv \alpha : *, \beta : *$

$$\frac{\Gamma \vdash \alpha : * \quad \Gamma \vdash \beta : *}{\Gamma \vdash \alpha \rightarrow \beta : *} \quad s \equiv *$$

$$\frac{\Gamma \vdash * : \square \quad \Gamma \vdash * : \square}{\Gamma \vdash * \rightarrow * : \square} \quad s \equiv \square$$

■ Not an instance

$\alpha \rightarrow *$ $\alpha : *$ but $* : \square$, and the two sorts must be the same

The rules (*appl*) and (*abst*) of $\lambda\omega$

$$\frac{\Gamma \vdash M : A \rightarrow B \quad \Gamma \vdash N : A}{\Gamma \vdash MN : B} \text{ (appl)} \qquad \frac{\Gamma, x:A \vdash M : B \quad \Gamma \vdash A \rightarrow B : s}{\Gamma \vdash \lambda x:A. M : A \rightarrow B} \text{ (abst)}$$

The second premiss of (*abst*) checks that the arrow type is well-formed. Both rules serve twice: for terms, with $s \equiv *$, and for type constructors, with $s \equiv \square$.

■ Instance, for a term

$$\frac{\alpha : *, x : \alpha \vdash x : \alpha \quad \alpha : * \vdash \alpha \rightarrow \alpha : *}{\alpha : * \vdash \lambda x : \alpha. x : \alpha \rightarrow \alpha} \text{ (abst), } s \equiv *$$

N&G §4.5

A typing problem in $\lambda\omega$

Problem

Find a kind κ with $\beta : * \vdash (\lambda\alpha : *. \alpha \rightarrow \alpha)\beta : \kappa$.

(a)	$\beta : *$	
(b)	$\alpha : *$	
(1)	$\alpha \rightarrow \alpha : *$	(<i>form</i>) on (b), (b)
(2)	$\lambda\alpha : *. \alpha \rightarrow \alpha : * \rightarrow *$	(<i>abst</i>) on (1)
(3)	$(\lambda\alpha : *. \alpha \rightarrow \alpha)\beta : *$	(<i>appl</i>) on (2), (a)

- In (1), α is a type, so $\alpha \rightarrow \alpha$ is a type.
- In (2), closing the flag of α puts its type in front, giving the kind $* \rightarrow *$.
- In (3), the argument β has kind $*$, so the application has kind $*$. It is a type.

The steps by (*sort*), (*var*) and (*weak*), and the second premiss of (*abst*), are routine.

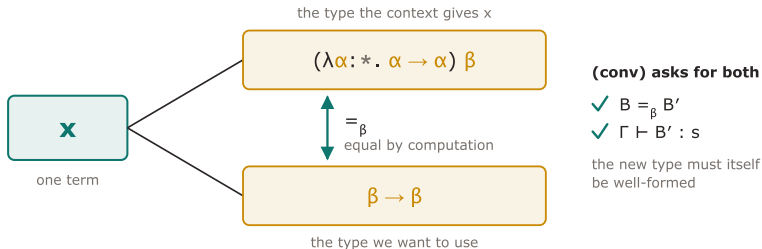
Derivations leave them out from here on.

N&G §4.5, §4.6

The rule (*conv*)

$$\frac{\Gamma \vdash A : B \quad \Gamma \vdash B' : s}{\Gamma \vdash A : B'} \text{ (conv) if } B =_{\beta} B'$$

If A has type B , it also has every well-formed type B' that is β -convertible to B . Without this rule a variable $x : (\lambda\alpha:*. \alpha \rightarrow \alpha)\beta$ would not have type $\beta \rightarrow \beta$: no other rule looks inside a type.



The second premiss of (*conv*)

$B =_{\beta} B'$ alone does not make B' well-formed.

■ Not an instance, with $\Gamma \equiv \beta : *, \gamma : *, x : \beta \rightarrow \gamma$

$$\frac{\Gamma \vdash x : \beta \rightarrow \gamma \quad \Gamma \vdash (\lambda \alpha : *. \beta \rightarrow \gamma) x : *}{\Gamma \vdash x : (\lambda \alpha : *. \beta \rightarrow \gamma) x} \text{ the second premiss is not derivable}$$

The side condition holds: $\beta \rightarrow \gamma =_{\beta} (\lambda \alpha : *. \beta \rightarrow \gamma) x$, since α does not occur in $\beta \rightarrow \gamma$. But the constructor expects a type and x is a term, so the application has no kind. The second premiss is what rejects it.

N&G Rem. 4.7.2

Constructors and kinds, in Lean

Lean has no symbol λ : the kind $\text{Type} \rightarrow \text{Type}$ has type $\text{Type } 1$.

```
def SelfMap : Type → Type := fun α => α → α      --  $\lambda\alpha:*. \alpha \rightarrow \alpha$ 

-- level 1, a term
#check (fun n => n + 1 : SelfMap Nat)    -- fun n => n + 1 : Nat → Nat
-- level 2, a type, and a proper constructor
#check SelfMap Nat                      -- SelfMap Nat : Type
#check (SelfMap)                        -- SelfMap : Type → Type
-- level 3, a kind
#check Type → Type                      -- Type → Type : Type 1

-- Lean's library is full of proper constructors
#check (List : Type → Type)
#check (Prod : Type → Type → Type)
```

(conv) in Lean: definitional equality

Lean applies (conv) without being asked. Expressions that are equal by computation are called **definitionally equal**.

```
def SelfMap : Type → Type := fun α => α → α
```

```
#reduce (types := true) SelfMap Nat      -- Nat → Nat
```

```
-- declared at one spelling of the type, used at the other
```

```
def succ' : SelfMap Nat := fun n => n + 1
```

```
#check (succ' : Nat → Nat)              -- accepted
```

```
#eval succ' 3                            -- 4
```

```
-- the side condition of (conv), stated and proved
```

```
example : SelfMap Nat = (Nat → Nat) := rfl
```

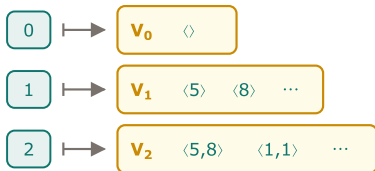
rfl proves $a = b$ exactly when a and b are definitionally equal.

λP : types depending on terms

λP adds abstraction over a term variable inside a type: $\lambda x:A. M$ with M a type.
Two uses, both of kind $\text{nat} \rightarrow *$.

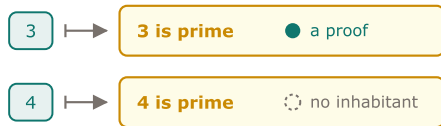
a family of types

$\lambda n:\text{nat}. V_n : \text{nat} \rightarrow *$



a predicate

$\lambda n:\text{nat}. P_n : \text{nat} \rightarrow *$



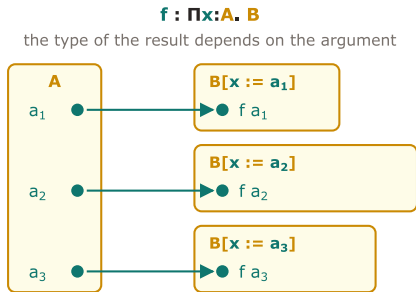
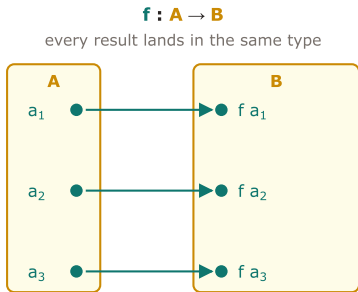
a function from terms to types: one type for every n

A proposition-valued function is what logic calls a predicate: the P in λP .

N&G §5.1

Π -types in λP

A function f with $f n : V_n$ for every $n : nat$ has no arrow type: the type of its result depends on its argument. Its type is $\Pi n:nat. V_n$.



if x does not occur in B , the three boxes are one box, and $\Pi x:A. B$ is $A \rightarrow B$

From here on $A \rightarrow B$ abbreviates $\Pi x:A. B$ with x not free in B .

N&G Notation 5.2.1

The derivation rules of λP

(*sort*), (*var*), (*weak*) and (*conv*) are as in $\lambda\omega$. The other three now use Π -types.

$$\frac{\Gamma \vdash A : * \quad \Gamma, x:A \vdash B : s}{\Gamma \vdash \Pi x:A. B : s} \text{ (form)}$$

$$\frac{\Gamma \vdash M : \Pi x:A. B \quad \Gamma \vdash N : A}{\Gamma \vdash MN : B[x := N]} \text{ (appl)}$$

$$\frac{\Gamma, x:A \vdash M : B \quad \Gamma \vdash \Pi x:A. B : s}{\Gamma \vdash \lambda x:A. M : \Pi x:A. B} \text{ (abst)}$$

s is $*$ or $\square$.

N&G Fig. 5.1

The rule (form) of λP

$$\frac{\Gamma \vdash A : * \quad \Gamma, x:A \vdash B : s}{\Gamma \vdash \Pi x:A. B : s} \text{ (form)}$$

B is typed with $x : A$ in the context, since it may mention x . The domain A must be a type: λP abstracts over terms only.

Instances, with $\Gamma \equiv A : *, P : A \rightarrow *$

$$\frac{A : * \vdash A : * \quad A : *, x:A \vdash * : \Box}{A : * \vdash A \rightarrow * : \Box} s \equiv \Box \qquad \frac{\Gamma \vdash A : * \quad \Gamma, x:A \vdash Px : *}{\Gamma \vdash \Pi x:A. Px : *} s \equiv *$$

Not an instance

$\Pi \alpha : *. \alpha \rightarrow \alpha$ the domain $*$ is a kind, not a type

From λP on, a kind is anything of type $\Box$. $A \rightarrow *$ is the kind of a predicate on A . N&G §5.2, §5.3

The rule (*appl*) of λP

$$\frac{\Gamma \vdash M : \Pi x:A. B \quad \Gamma \vdash N : A}{\Gamma \vdash MN : B[x := N]} \text{ (appl)}$$

The argument N is substituted for x in B : the type of the result depends on the argument.

■ Instances, with $\Gamma \equiv A : *$, $P : A \rightarrow *$, $a : A$, $f : \Pi x:A. Px$

$$\frac{\Gamma \vdash P : A \rightarrow * \quad \Gamma \vdash a : A}{\Gamma \vdash Pa : *} \qquad \frac{\Gamma \vdash f : \Pi x:A. Px \quad \Gamma \vdash a : A}{\Gamma \vdash fa : Pa}$$

On the left the result is a type, Pa . On the right it is a term of that type, since $(Px)[x := a] \equiv Pa$. N&G §5.2, §5.3

The rule (*abst*) of λP

$$\frac{\Gamma, x:A \vdash M : B \quad \Gamma \vdash \Pi x:A. B : s}{\Gamma \vdash \lambda x:A. M : \Pi x:A. B} \text{ (abst)}$$

The declaration $x : A$ leaves the context and becomes a binder: λ in the term, and Π in the type, since B may mention x . The second premiss checks that the Π -type is well-formed.

■ Instance, with $\Gamma \equiv A : *, P : A \rightarrow *, f : \Pi x:A. Px$

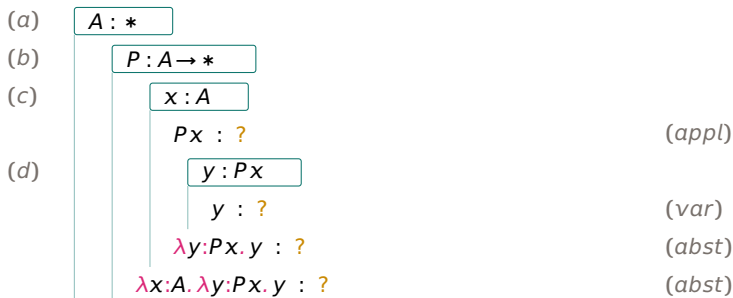
$$\frac{\Gamma, x:A \vdash fx : Px \quad \Gamma \vdash \Pi x:A. Px : *}{\Gamma \vdash \lambda x:A. fx : \Pi x:A. Px}$$

A typing problem in λP

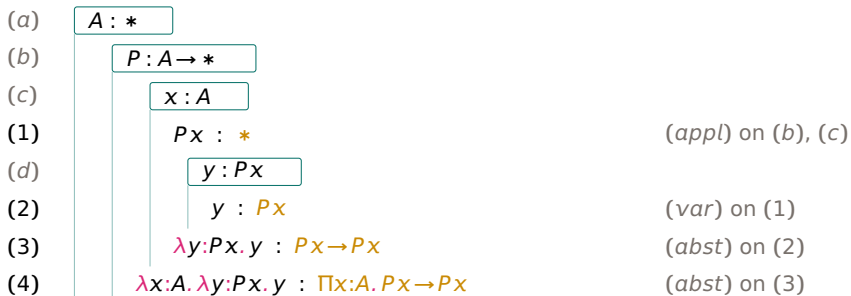
Problem

Find a type ρ with $A : *$, $P : A \rightarrow *$ $\vdash \lambda x:A. \lambda y:Px. y : \rho$.

Bottom to top, each λ opens a flag. The flag of y needs Px to be a type.



The derivation, from the inside out



- In (1), Px is a type, so a variable of that type may be declared: flag (d).
- In (3), the flag of y closes into an arrow, since Px does not mention y .
- In (4), the flag of x closes into a Π , since $Px \rightarrow Px$ mentions x .

Types depending on terms, in Lean

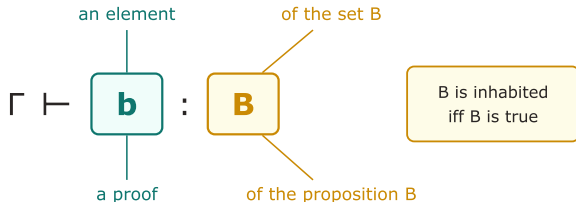
```
def Vec (α : Type) : Nat → Type          -- a family:  $\lambda n:nat. V_n$ 
  | 0      => Unit
  | n + 1 => α × Vec α n
#check Vec Nat                            -- Vec Nat : Nat → Type
#check ((3, 4, ())) : Vec Nat 2

def zeros : (n : Nat) → Vec Nat n        -- a term of a  $\Pi$ -type
  | 0      => ( )
  | n + 1 => (0, zeros n)
#check zeros 2                            -- zeros 2 : Vec Nat 2
#eval  zeros 2                           -- (0, 0, ( ))

def IsPrime (n : Nat) : Prop :=           -- a predicate:  $\lambda n:nat. P_n$ 
  2 ≤ n ∧ ∀ d, d | n → d = 1 ∨ d = n
#check (IsPrime)                          -- IsPrime : Nat → Prop
(0, zeros n) is accepted at Vec Nat (n + 1), which computes to Nat × Vec Nat n:
(conv) again.
```

Propositions as types, proofs as terms

read as mathematics



read as logic

The PAT-interpretation

A proposition is coded as a type. If $b : B$ and B codes a proposition, then b is a **proof object** for B . If B has no inhabitant, B has no proof.

To claim that B is true, one must exhibit a context Γ and a term b with $\Gamma \vdash b : B$, and that judgement can be checked.

N&G §5.4

Minimal predicate logic

The logic that λP will code. It speaks about:

- **sets** S , with elements $a \in S$;
- **propositions** A, B ;
- **predicates** P on a set S : for each $x \in S$, $P(x)$ is a proposition.

Propositions are combined by two operations only:

$A \Rightarrow B$ implication $\forall_{x \in S} (P(x))$ universal quantification

Minimal means that there is no $\neg$, $\wedge$, $\vee$ or $\exists$.

■ Examples, for a predicate P and a binary predicate Q on S

$$\forall_{x \in S} (P(x) \Rightarrow P(x)) \qquad \forall_{x \in S} \forall_{y \in S} (Q(x, y)) \Rightarrow \forall_{u \in S} (Q(u, u))$$

N&G §5.4

Natural deduction: elimination and introduction rules

A proof system for this logic. Each operation has an **elimination** rule, saying how to use it, and an **introduction** rule, saying how to prove it.

$$\frac{A \Rightarrow B \quad A}{B} (\Rightarrow\text{-elim})$$

from $A \Rightarrow B$ and A , conclude B

$$\frac{\text{Assume } A \cdots B}{A \Rightarrow B} (\Rightarrow\text{-intro})$$

if B follows from assuming A , conclude $A \Rightarrow B$

$$\frac{\forall_{x \in S} (P(x)) \quad N \in S}{P(N)} (\forall\text{-elim})$$

what holds for all x holds for a given N

$$\frac{\text{Let } x \in S \cdots P(x)}{\forall_{x \in S} (P(x))} (\forall\text{-intro})$$

what holds for an arbitrary x holds for all

“Assume A ” and “Let $x \in S$ ” are flags, as in L2: each is in force only up to the line where its rule is applied.

N&G Ex. 2.4.9, §5.4

Why $\Rightarrow$ is $\rightarrow$, and $\forall$ is Π

Neither coding is a choice. Each follows from “true means inhabited”.

Implication

$A \Rightarrow B$ is true
iff if A is true, then B is true
iff if A is inhabited, then B is inhabited
iff there is a function mapping
inhabitants of A to inhabitants of B
iff $A \rightarrow B$ is inhabited.

Universal quantification

$\forall_{x \in S} (P(x))$ is true
iff for each x in S , $P(x)$ is true
iff for each x in S , Px is inhabited
iff there is a function mapping each $x : S$
to an inhabitant of Px
iff $\Pi x:S. Px$ is inhabited.

The second function has an output type that depends on its input: predicate logic needs λP , while implication alone was already in $\lambda \rightarrow$. N&G §5.4

($\forall$ -elim) is (*appl*)

$$\frac{\forall_{x \in S} (P(x)) \quad N \in S}{P(N)} \quad (\forall\text{-elim})$$

$$\frac{\Gamma \vdash M : \Pi x:A. B \quad \Gamma \vdash N : A}{\Gamma \vdash MN : B[x := N]} \quad (\text{appl})$$

Read S as A , $P(x)$ as B , $\forall$ as Π . Then (*appl*) says what ($\forall$ -elim) says. In addition it names the context, and it carries the proof objects, in teal.

■ Instance, with $\Gamma \equiv S : *$, $P : S \rightarrow *$, $a : S$, $h : \Pi x:S. Px$

$$\frac{\Gamma \vdash h : \Pi x:S. Px \quad \Gamma \vdash a : S}{\Gamma \vdash ha : Pa}$$

If h proves $\forall_{x \in S} (P(x))$ and a is an element of S , then ha proves $P(a)$. ($\Rightarrow$ -elim)
is the case where B does not mention x . N&G §5.4

($\forall$ -intro) is (*abst*)

$$\frac{\text{Let } x \in S \cdots P(x)}{\forall_{x \in S} (P(x))} \quad (\forall\text{-intro}) \qquad \frac{\Gamma, x:A \vdash M : B}{\Gamma \vdash \lambda x:A. M : \Pi x:A. B} \quad (\text{abst})$$

The flag “Let $x \in S$ ” is the declaration $x : A$. Closing the flag abstracts the proof object.

■ Instance, with $\Gamma \equiv A : *, P : A \rightarrow *$

$$\frac{\Gamma, x:A \vdash \lambda y:P x. y : P x \rightarrow P x}{\Gamma \vdash \lambda x:A. \lambda y:P x. y : \Pi x:A. P x \rightarrow P x}$$

For an arbitrary x , the term $\lambda y:P x. y$ proves $P(x) \Rightarrow P(x)$. So the abstraction proves $\forall_{x \in A} (P(x) \Rightarrow P(x))$.

The second premiss of (*abst*) is left out here. ($\Rightarrow$ -intro) is the case where B does not mention x .

N&G §5.4, Rem. 5.4.3

Minimal predicate logic, coded in λP

minimal predicate logic	the type theory of λP
S is a set	$S : *$
A is a proposition	$A : *$
$a \in S$	$a : S$
p proves A	$p : A$
P is a predicate on S	$P : S \rightarrow *$
$A \Rightarrow B$	$A \rightarrow B \quad (\equiv \Pi x:A. B)$
$\forall_{x \in S} (P(x))$	$\Pi x:S. P x$
$(\Rightarrow\text{-elim}), (\forall\text{-elim})$	$(appl)$
$(\Rightarrow\text{-intro}), (\forall\text{-intro})$	$(abst)$

N&G Fig. 5.2

theorem is def

The same term, once as a function on data and once as a proof. Lean's sort of propositions is Prop; a proposition without a proof is a type without an inhabitant.

```
def      idData (A : Type) : A → A := fun a => a
theorem idProp (A : Prop)  : A → A := fun a => a

#print idData
-- def idData : (A : Type) → A → A := fun A a => a
#print idProp
-- theorem idProp : ∀ (A : Prop), A → A := fun A a => a

#print False
-- inductive False : Prop
-- number of parameters: 0
-- constructors:
```

Π -types, in Lean

All five are Π -types. Lean chooses how to print each one.

```
-- typed as (x : A) → B
#check (α : Type) → α → α
#check (n : Nat) → Nat
#check (n : Nat) → n = n

-- (α : Type) → α → α : Type 1
-- Nat → Nat : Type
-- ∀ (n : Nat), n = n : Prop

-- typed as ∀ x : A, B
#check ∀ α : Type, α → α
#check ∀ n : Nat, n = n

-- (α : Type) → α → α : Type 1
-- ∀ (n : Nat), n = n : Prop
```

on paper	Lean prints	when
$\Pi x:A. B$	$(x : A) \rightarrow B$	B mentions x
$A \rightarrow B$	$A \rightarrow B$	B does not mention x
$\Pi x:A. B$	$\forall (x : A), B$	B is a proposition

Using and proving $\forall$ and $\Rightarrow$, in Lean

```
variable (S : Type) (P : S → Prop)
```

```
-- ( $\forall$ -elim) is (appl): apply the proof to the element
```

```
example (h :  $\forall$  x : S, P x) (a : S) : P a := h a
```

```
-- ( $\forall$ -intro) is (abst): abstract over an arbitrary element
```

```
example (h :  $\forall$  x : S, P x) :  $\forall$  y : S, P y := fun y => h y
```

```
-- ( $\Rightarrow$ -elim) and ( $\Rightarrow$ -intro) are the same two rules
```

```
example (A B : Prop) (f : A → B) (a : A) : B := f a
```

```
example (A : Prop) : A → A := fun a => a
```

Nothing was added to the language for quantifiers or implication: application and fun are the whole of it.

A proving problem in λP

Problem

For a set S and a binary predicate Q on S , prove

$$\forall_{x \in S} \forall_{y \in S} (Q(x, y)) \Rightarrow \forall_{u \in S} (Q(u, u)).$$

Coded in λP : in the context $S : *$, $Q : S \rightarrow S \rightarrow *$, find a term of type $H \rightarrow \prod u:S. Q u u$, where $H \equiv \prod x:S. \prod y:S. Q x y$. This time the unknown is the term.



The binary predicate is Curried.

N&G §5.5

One derivation, read twice

	natural deduction	λP , with $H \equiv \prod x:S. \prod y:S. Q x y$	
(a)	Assume $\forall_{x \in S} \forall_{y \in S} (Q(x, y))$	$z : H$	
(b)	Let $u \in S$	$u : S$	
(1)	$\forall_{y \in S} (Q(u, y))$	$z u : \prod y:S. Q u y$	($\forall$ -elim) = (<i>appl</i>)
(2)	$Q(u, u)$	$z u u : Q u u$	($\forall$ -elim) = (<i>appl</i>)
(3)	$\forall_{u \in S} (Q(u, u))$	$\lambda u:S. z u u : \prod u:S. Q u u$	($\forall$ -intro) = (<i>abst</i>)
(4)	$\forall_x \forall_y (Q(x, y)) \Rightarrow \forall_u (Q(u, u))$	$\lambda z:H. \lambda u:S. z u u : H \rightarrow \prod u:S. Q u u$	($\Rightarrow$ -intro) = (<i>abst</i>)

- In (1), the argument u is substituted for x in the type of z .
- Same flags, same steps. The reading on the right also records the proof object, in teal.
- From the term of line (4) alone, the whole derivation can be rebuilt.

The proof object, in Lean

The term of line (4). Lean is given the proposition and the term, and checks the one against the other.

```
theorem diag (S : Type) (Q : S → S → Prop) :  
  (∀ x y : S, Q x y) → ∀ u : S, Q u u :=  
  fun z u => z u u
```

```
#check diag  
-- diag (S : Type) (Q : S → S → Prop) :  
--   (∀ (x y : S), Q x y) → ∀ (u : S), Q u u
```

This is Type Checking, with a proposition in the place of the type.

λC : all four dependencies

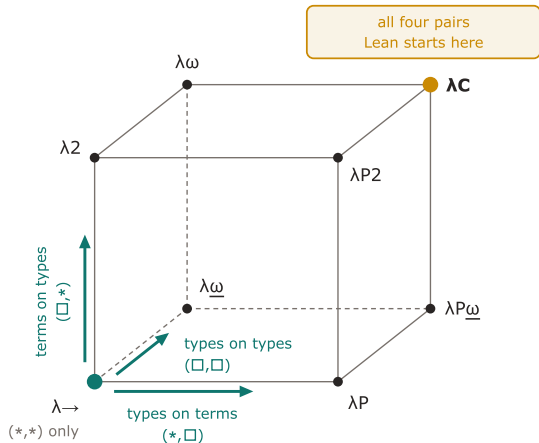
$\lambda\omega$ forms $A \rightarrow B$ with A and B of the same sort. λP forms $\Pi x:A. B$ with $A : *$. λC , the Calculus of Constructions, lets the two sorts be chosen independently.

$$\frac{\Gamma \vdash A : s_1 \quad \Gamma, x:A \vdash B : s_2}{\Gamma \vdash \Pi x:A. B : s_2} \text{ (form)}$$

The Π -type has the sort of its body, s_2 . For an abstraction $\lambda x:A. b$ with $b : B$, the pair (s_1, s_2) says what depends on what.

(s_1, s_2)	$\lambda x:A. b$ is a	from	a Π -type formed with this pair
$(*, *)$	term depending on a term	$\lambda \rightarrow$	$nat \rightarrow nat : *$
$(\square, *)$	term depending on a type	$\lambda 2$	$\Pi \alpha : *. \alpha \rightarrow \alpha : *$
$(\square, \square)$	type depending on a type	$\lambda \omega$	$* \rightarrow * : \square$
$(*, \square)$	type depending on a term	λP	$nat \rightarrow * : \square$

The λ -cube



The three extensions are independent: three perpendicular directions. Each adds one pair to $(*, *)$.

$\lambda \rightarrow$ $(*, *)$

$\lambda 2$ $(*, *)$ ($\square, *$)

$\lambda \omega$ $(*, *)$ ($\square, \square$)

λP $(*, *)$ ($*, \square$)

$\lambda P 2$ $(*, *)$ ($\square, *$) ($*, \square$)

λC $(*, *)$ ($\square, *$) ($\square, \square$) ($*, \square$)

Eight systems, one set of rules: only the pairs admitted by (*form*) differ.

N&G Fig. 6.1–6.3

The derivation rules of λC

Terms, types and kinds are one syntax:

$\mathcal{E} = \mathbb{V} \mid \square \mid * \mid (\mathcal{E} \mathcal{E}) \mid (\lambda \mathbb{V} : \mathcal{E}. \mathcal{E}) \mid (\Pi \mathbb{V} : \mathcal{E}. \mathcal{E})$.

$$\begin{array}{c}
 \frac{}{\emptyset \vdash * : \square} \text{ (sort)} \quad \frac{\Gamma \vdash A : s}{\Gamma, x:A \vdash x : A} \text{ (var)} \quad \frac{\Gamma \vdash A : B \quad \Gamma \vdash C : s}{\Gamma, x:C \vdash A : B} \text{ (weak)} \\
 \\
 \frac{\Gamma \vdash A : s_1 \quad \Gamma, x:A \vdash B : s_2}{\Gamma \vdash \Pi x:A. B : s_2} \text{ (form)} \quad \frac{\Gamma \vdash A : B \quad \Gamma \vdash B' : s}{\Gamma \vdash A : B'} \text{ (conv) if } B =_{\beta} B' \\
 \\
 \frac{\Gamma \vdash M : \Pi x:A. B \quad \Gamma \vdash N : A}{\Gamma \vdash MN : B[x := N]} \text{ (appl)} \quad \frac{\Gamma, x:A \vdash M : B \quad \Gamma \vdash \Pi x:A. B : s}{\Gamma \vdash \lambda x:A. M : \Pi x:A. B} \text{ (abst)}
 \end{array}$$

$(abst_2)$ and $(appl_2)$ are $(abst)$ and $(appl)$ with $A \equiv *$.

N&G Fig. 6.4, Def. 6.3.1

A typing problem in λC

Problem

Find ρ with $\emptyset \vdash \lambda A:*. \lambda P:A \rightarrow *. M : \rho$, for the $M \equiv \lambda x:A. \lambda y:P x. y$ of λP .

- (a) $A : *$
- (b) $P : A \rightarrow *$
- (1) $M : T$ as derived in λP
- (2) $\lambda P:A \rightarrow *. M : \Pi P:A \rightarrow *. T$ (*abst*) on (1)
- (3) $\lambda A:*. \lambda P:A \rightarrow *. M : \Pi A:*. \Pi P:A \rightarrow *. T$ (*abst*) on (2)

Here $T \equiv \Pi x:A. Px \rightarrow Px$. Each (*abst*) needs its Π -type to be formed, and each formation uses one pair:

$\Pi x:A. \dots$ and $Px \rightarrow Px$	domain : $*$, body : $*$	$(*, *)$
$A \rightarrow *$	domain : $*$, body : $\square$	$(*, \square)$
$\Pi P:A \rightarrow *. \dots$ and $\Pi A:*. \dots$	domain : $\square$, body : $*$	$(\square, *)$

Three pairs, so the derivation lives in $\lambda P2$.

N&G §6.1, Fig. 6.2

Which corner is Lean?

All four pairs, so Lean starts from λC . Lean's sorts form a ladder, $\text{Prop} : \text{Type} : \text{Type}$. 1: Prop and Type both play $*$, and the sort above plays $\square$. Each line ends with the pair it uses.

```
#check Nat → Nat           -- : Type           (*, *)
#check ∀ p : Prop, p → p    -- : Prop          (□, *)
#check fun (α : Type) => α → α -- : Type → Type  (□, □)
#check fun (n : Nat) => n = n -- : Nat → Prop    (*, □)
```

-- (□, *) over Prop stays in Prop; over Type it climbs one step

```
#check ∀ α : Type, α → α    -- : Type 1
```

In $\lambda 2$ the type $\Pi \alpha : *. \alpha \rightarrow \alpha$ quantifies over all types and is itself one of them: it is **impredicative**. Lean keeps this for Prop only. N&G §3.8

What λC guarantees

The properties of $\lambda \rightarrow$ survive all three extensions. Four of them are what a proof checker stands on.

property	statement	for a checker
Uniqueness of Types	if $\Gamma \vdash A : B_1$ and $\Gamma \vdash A : B_2$, then $B_1 =_\beta B_2$	the type of a term can be computed
Church–Rosser	if $M =_\beta N$, then M and N reduce to a common L	$B =_\beta B'$ can be tested by reducing
Subject Reduction	if $\Gamma \vdash A : B$ and $A \rightarrow_\beta A'$, then $\Gamma \vdash A' : B$	reducing never breaks a typing
Strong Normalisation	every legal expression is strongly normalising	the reducing always stops

The proofs are long; for Strong Normalisation see Barendregt (1992). N&G Lem. 6.3.9, Cor. 6.3.12, Lem. 6.3.13, Thm. 6.3.14

Checking is decidable, finding is not

1	$\boxed{?} \vdash M : \boxed{?}$	Well-typedness is M legal: does it have a type at all? in all eight systems of the cube	✓ decidable
2	$\Gamma \vdash M \overset{?}{:} A$	Type Checking is M a proof of A? in all eight systems of the cube	✓ decidable
3	$\Gamma \vdash \boxed{?} : A$	Term Finding is A provable? decidable only in $\lambda \rightarrow$ and $\lambda \underline{\omega}$	✗ undecidable

Theorem

In λC and its subsystems, Well-typedness and Type Checking are decidable.
Term Finding is decidable in $\lambda \rightarrow$ and $\lambda \underline{\omega}$ only.

Finding a term of type A is proving A , and no algorithm decides provability: Church and Turing, 1936 (Church 1936).
N&G Thm. 6.3.15

The three problems, in Lean

```
variable (S : Type) (Q : S → S → Prop)
```

-- 1. Well-typedness: from the term alone, Lean computes a type

```
#check fun (z : ∀ x y, Q x y) (u : S) => z u u
```

```
-- fun z u => z u u : (∀ (x y : S), Q x y) → ∀ (u : S), Q u u
```

-- 2. Type Checking: from the term and the type, Lean answers yes or no

```
example : (∀ x y, Q x y) → ∀ u, Q u u := fun z u => z u u -- yes
```

```
#guard_msgs (drop error) in
```

```
example : (∀ x y, Q x y) → ∀ u, Q u u := fun z u => z u -- no
```

```
-- error: z u has type ∀ (y : S), Q u y, but Q u u is expected
```

-- 3. Term Finding: from the type alone, Lean does not search

```
#guard_msgs (drop error) in
```

```
example : (∀ x y, Q x y) → ∀ u, Q u u := _
```

```
-- error: don't know how to synthesize placeholder
```

The refusal in 2 names the step that fails: line (1) of “One derivation, read twice”.

λC is still bare

λC can express the mathematics. Writing it down in λC is another matter:

missing in λC	consequence	added by
$\neg, \wedge, \vee, \exists$	only coded, through Π over all propositions: $A \wedge B \equiv \Pi C: * . (A \rightarrow B \rightarrow C) \rightarrow C$	the codings, and their rules derived
names	every use of a notion repeats its full term; no def, no theorem	definitions
data types with induction	numbers can be coded too, by a Π over all types, but the coding yields no induction principle	inductive types

■ Next lecture

The connectives, coded: $\perp, \neg, \wedge, \vee, \exists$ as Π -types, and their rules derived.

References I

Avigad, J., L. de Moura, S. Kong, and S. Ullrich (2026). *Theorem Proving in Lean 4*. With contributions from the Lean community; tested against Lean 4.33.0. URL: https://lean-lang.org/theorem_proving_in_lean4/ (visited on 08/20/2026).

Barendregt, H. P. (1992). “Lambda calculi with types”. In: *Handbook of Logic in Computer Science*. Ed. by S. Abramsky, D. M. Gabbay, and T. S. E. Maibaum. Vol. 2. Oxford University Press, pp. 117–309.

Bruijn, N. G. de (1970). “The mathematical language AUTOMATH, its usage, and some of its extensions”. In: *Symposium on Automatic Demonstration*. Vol. 125. Lecture Notes in Mathematics. Springer, pp. 29–61.

Church, A. (1936). “A note on the Entscheidungsproblem”. In: *Journal of Symbolic Logic* 1.1, pp. 40–41.

Coquand, T. and G. Huet (1988). “The calculus of constructions”. In: *Information and Computation* 76.2–3, pp. 95–120.

References II

Howard, W. A. (1980). “The formulae-as-types notion of construction”. In: *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*. Ed. by J. P. Seldin and J. R. Hindley. Academic Press, pp. 479–490.

Nederpelt, R. and H. Geuvers (2014a). “Second order typed lambda calculus”. In: *Type Theory and Formal Proof: An Introduction*. Cambridge: Cambridge University Press. Chap. 3, pp. 69–84.

— (2014b). “The Calculus of Constructions”. In: *Type Theory and Formal Proof: An Introduction*. Cambridge: Cambridge University Press. Chap. 6, pp. 123–136.

— (2014c). *Type Theory and Formal Proof: An Introduction*. Cambridge: Cambridge University Press.

— (2014d). “Types dependent on terms”. In: *Type Theory and Formal Proof: An Introduction*. Cambridge: Cambridge University Press. Chap. 5, pp. 103–122.

References III

Nederpelt, R. and H. Geuvers (2014e). “Types dependent on types”. In: *Type Theory and Formal Proof: An Introduction*. Cambridge: Cambridge University Press. Chap. 4, pp. 85–102.

Chapters 3 to 6 of the book throughout. Ch. 3: §3.1 (Ex. 3.1.1), §3.2 Π -types, §3.3 (Def. 3.3.1, 3.3.2), §3.4 (Def. 3.4.1, 3.4.4, 3.4.7, Ex. 3.4.5, Fig. 3.1), §3.5 the derivation, §3.6 (Def. 3.6.2), §3.8 impredicativity and the history. Ch. 4: §4.1 (Def. 4.1.4, 4.1.6, Rem. 4.1.7), §4.2–4.4 (Def. 4.2.1, 4.2.2, 4.3.1, 4.4.1), §4.5 the derivation, §4.6 shortened derivations, §4.7 (Def. 4.7.1, Rem. 4.7.2), Fig. 4.1. Ch. 5: §5.1, §5.2 (Fig. 5.1, Notation 5.2.1), §5.3 the derivation, §5.4 (Rem. 5.4.3, Fig. 5.2), §5.5 the worked example, §5.7 the history. Ch. 6: §6.1, §6.2 (Fig. 6.1–6.4), §6.3 (Def. 6.3.1, Lem. 6.3.9, Cor. 6.3.12, Lem. 6.3.13, Thm. 6.3.14, 6.3.15), §6.5. From other chapters: Ex. 2.4.9 (natural deduction) and §7.2. The typing problem in λC is not from the book. The Lean slides follow *Theorem Proving in Lean 4*: ch. 2 (dependent type theory, `#reduce`) and ch. 3 (propositions as types).